User Menu for Currency Denomination Calculator

Requirements/Specification:

This program is tasked to allow the user to input any sets of information of a person's name and money values associated with it. Upon retrieving the data from the user, the program displays a menu for the user to choose from.
The choices are:

1. User may search for a name and display its name, money, and denominations
2. Find the name with the smallest amount and display the denomination breakdown.
3. Find the name with the largest amount and display the denomination breakdown.
4. Calculate and display the total number of currency notes for each denomination.
5. Calculate and display the total sum of all denominations.
6. Exit.

After the user makes a choice, the menu is re-displayed again for the user to use.

Assumptions:

1) User inputs multiples of 5 for money
2) The maximum number of "Person" corresponds to the array's capacity

Conditions:

1) To use preset data, usePreset must be "true".
2) To manually input data, usePreset must be "false"

User Guide: Instructions on running the Program

1. Save file under ICT167 Assignment -> assignment (package name) -> ChangeC (class name) and Person (class name) for the respective codes.
2. Upload the codes to Apache NetBeans IDE 19
3. Before running the code, ensure that the decision to use preset data or manually input data is selected by assigning usePreset to true (preset) or false (manual).
4. Next, run the program and expect to see the user menu to interact with if the use of preset data is selected
5. Alternatively, if the user decides to manually input data, he/she will be prompted to input names and money values of person and after each set of name and money values, the user is prompted with an option to add more (up to 10 persons). Once the user has finished inputting the data, the user menu will be available for interaction.
6. The user is prompt with a menu of 6 options:
    - Option 1: Enter a name and display change to be given for each denomination.
    - Option 2: Find the name with the smallest amount and display change to be given for each denomination.
    - Option 3: Find the name with the largest amount and display change to be given for each denomination.
    - Option 4: Calculate and display the total number of currency notes for each denomination.
    - Option 5: Calculate and display the total sum of all denominations.
    - Option 6: Exit
7. Upon making the choice, the system will display the desired information to the user and prompt the menu again, allowing the user to have another interaction with the system.

Structure/Design/Algorithm:

- Structure: Method Functions
    1) getPersonName – Used to prompt user to input the name of the person and store its data. Also, if a duplicate name has been detected, the system will prompt that the name has already been registered into the system.

    2) getPersonMoney - Used to prompt user to input the money of the name and store its data. Also, if the money input is not a multiple of 5, the system will prompt that the value is input and requires the user to input a new and valid value.

    3) denomination – Used to breakdown the money of the respective name that has been chosen into the different denominations provided (denominations = 1000, 500, 200,100, 50, 20, 10)

    4) currencyMenu – menu for the user to choose the functions of the program they wish to use

    5) getSample – Used to hardcode a preset data for testing the interaction between the user and currencyMenu

    6) printInfo – Used to check the getSample hardcoded data by printing

    7) StudentInfo – Used to print my info (Name, Student Number, School, Lecturer's Name)

- Design: Written Description and UML Class Diagram

The code consists of two main classes: “Person” and “ChangeC”.

The “Person” class stores a person object containing a name and an amount of money. The “ChangeC’ class handles the logic for inputting data, calling of methods and the main program that interacts with the user

“Person” class has:

Attributes:

- name: A String representing the person's name.
- money: A double representing the amount of money the person has.

Methods:

- Constructor (Person): Initializes the name and money attributes.
- getName: Returns the person's name.
- getMoney: Returns the person's money.
- setName: Sets the person's name.
- setMoney: Sets the person's money.

“ChangeC” class has:

Attributes:

- arrayCapacity: maximum number of Person objects in an array.
- people: An array of Person objects.
- usePreset: allows the user to state their decision of using preset data (boolean)

Methods:

- main: the primary part of the code that calls methods from the rest of code to run the program
- getPersonName: prompts the user to enter a name, to store in array
- getPersonMoney: prompts the user to enter an amount of money, to store in array and ensuring input is a multiple of 5
- denomination: Calculates and prints a breakdown of money into each denominations provided
- currencyMenu: Displays a menu allowing the user to choose between options for different operations, such as displaying denominations for a specific person, finding the person with the smallest or largest amount, calculating total sum and amount of denominations
- getSample: hardcodes some preset data in the array for testing purposes.
- printInfo: prints data about all Person objects in the people array for testing purposes.
- StudentInfo: Prints student information

Person
- name: String - money: double
+ Person(name: String, money: double) + getName(): String + getMoney(): double + setName(name: String): void + setMoney(money: double): void + toString(): String |

ChangeC
- arrayCapacity: int - usePreset: boolean - index: int - cont: boolean - denominations: double - remainder: double - counter: int - answer: int - found: boolean - totalSum: double

```
+ main(args: String[]): void
+ getPersonName(input: Scanner, person: Person[]): String
+ getPersonMoney(input: Scanner): double
+ denomination(money: double): void
+ currencyMenu(input: Scanner, people: Person[]): void
+ getSample(data: Person[]): void
+ printInfo(person: Person[]): void
+ StudentInfo(): void
```

- Algorithm:

  Pseudocode for ChangeC Class:

```
studentInfo()
Scanner input
Person [] person = new Person[]
boolean usePreset = true

if(usePreset == true)
        getSample()
        printInfo ()
else
   Let int index = 0
   Let boolean cont = true
         while(cont == true && index < arr.length)
           String name = getPersonName()
           double money = getPersonMoney()
           Input next line

           Person newPerson = new Person()
           newPerson.setName()
           newPerson.setMoney()
           people[index++] = newPerson;

           if (index < arrayCapacity)
                print("Do you have more person to enter (Y/N)")
```

```
            String reply = input next line

        if(reply = n || reply = N)
            cont = false
      currencyMenu()
```

StudentInfo()

```
print("Student Name: Styverson Ng")

print ("Student Number: 35427675")

print ("Murdoch University")

print ("Lecturer's Name: Siew Cheong Chong")
```

getPersonName()

```
        print("Please enter the name of the person: ")
        String name = input next line()

         for (Person personIndex : person)
           if (personIndex != null && personIndex.getName() = (name))
              print("You have entered a registered name. Please enter a different name.")
              return getPersonName(input, person)
        return name
```

getPersonMoney()

```
        print("Please enter the currency note value for the person: ")
    while (true)
        if (input != double)
            print("Invalid input. Please enter a valid money value.")
            Input next

        else
            money = input next double
            Input next line

        if (money % 5 != 0)
           print("Please enter the amount in multiple of 5.")

        else
            return money
```

denomination(double money)

```
double initialAmount = money
remainder = 0
print (“AED 1000” + initialAmount/1000)
    remainder = initialAmount % 1000
print (“AED 500” + remainder/500)
    remainder = remainder % 500
print (“AED 200” + remainder/200)
    remainder = remainder % 200
print (“AED 100” + remainder/100)
    remainder = remainder % 100
print (“AED 50” + remainder/50)
    remainder = remainder % 50
print (“AED 20” + remainder/20)
    remainder = remainder % 20
print (“AED 10” + remainder/10)
```

currencyMenu()

```
Scan for user keyboard input
Let answer = input

print("Please enter your desired function: ")
print("1. Enter a name and display change to be given for each denomination. ")
print("2. Find the name with the smallest amount and display denomination
breakdown. ")
print("3. Find the name with the largest amount and display denomination
breakdown.")
print("4. Calculate and display the total number of notes for each denomination.")
print("5. Calculate and display the total amount for the sum of all denominations. ")
print("6. Exit ")

If(answer == 1)
ask for name
print (“Please enter name: ”)
Scan for user keyboard input
Search for input name in the array
```

Display details and call for denomination method

```
If(answer == 2)
Let smallest = 0
Find person with the smallest money amount
For(int i = 0; i < arr.length; i++)
If(currentMoney < smallest)
Smallest = currentMoney;
Display the person's details and call denomination method

If(answer == 3)
Let largest = 0
Find person with the largest money amount
For(int i = 0; i < arr.length; i++)
If(currentMoney > largest)
Largest = currentMoney
Display the person's details and call denomination method.

If(answer == 4)
Let total sum = 0
For(int i = 0; i < arr.length; i++)
If( money != null)
Totalsum += money

call denomination method

If(answer == 5)
Let totalSum = 0
For(int i = 0; i < arr.length; i++)
If( money != null)
totalSum += money

print(totalSum)

If(answer == 6)
break
```

getSample()

```
    Let data[0] = new Person("Charles",4685)
    Let data[1] = new Person("Carlos",3265)
    Let data[2] = new Person("Max",6985)
    Let data[3] = new Person("Pierre",6635)
    Let data[4] = new Person("Daniel",10180)
    Let data[5] = new Person("Fernando",1925)
    Let data[6] = new Person("Lewis",685)
    Let data[7] = new Person("Oscar",3215)
    Let data[8] = new Person("Lando",7345)
    Let data[9] = new Person("George",965);
}
```

printInfo()

```
For(int i = 0; i < arr.length; i++)
if (data[index] != null)
        System.out.println(data[index])
```

Design Concepts/Software Design Patterns

- **Open/Closed Principle & Encapsulation**

```java
package assignment;


public class Person {
    private String name;
    private double money;

    public Person(String personName, double personMoney){
        name = personName;
        money = personMoney;
    }

    public String getName(){
    return name;
    }

    public double getMoney(){
    return (int)money;
    }

    public void setName(String name) {
        this.name = name;
    }

    public void setMoney(double money) {
        this.money = money;
    }

    @Override
    public String toString(){
        return name + " AED " + money;
    }
}
```

→ The `Person` class can be extended without modifying the existing implementation.

→ The `Person` class bundles the data (name and money) with methods to manipulate that data.

- **Information Hiding & Private Variables:**

```java
    private String name;
    private double money;

```

→ Private variables (name and money) hide implementation details from the outside, and ensuring they can only be accessed through the class's public methods

- **Pre-conditions and Post-conditions:**

```java
    // method for inputting customer's money
    public static double getPersonMoney(Scanner input){
        System.out.println("Please enter the currency note value for the person: ");

        // loop to check if user input valid money value
        while (true){
            if (!input.hasNextDouble()){
                System.out.println("Invalid input. Please enter a valid money value.");
                input.next();
            }
            else{
                double money = input.nextDouble();
                    input.nextLine();

                // check and prompt user if money value is a multiple of 5
                if (money % 5 != 0) {
                    System.out.println("Please enter the amount in multiple of 5.");
                }
                else{
                    return money;
                }
            }
        }
    }
```

→ getPersonMoney is a method that enforces a pre-condition (valid input) to be true before execution and a post-condition (money as a multiple of 5) that must be true after the method is executed.

- **Constructors:**

```
public Person(String personName, double personMoney){
    name = personName;
    money = personMoney;
}
```

→ This is a constructor that initializes “name” and “money” variables when a “Person” object is created.

- **Accessors and Mutators:**

```
public String getName(){
return name;
}

public double getMoney(){
return (int)money;
}

public void setName(String name) {
    this.name = name;
}

public void setMoney(double money) {
    this.money = money;
}
```

→ These are the accessors (getters) and mutators (setters) of my class.

- **Helper Methods:**

```java
// method for calculating the denominations of customer's money
public static void denomination(double money){
    double[] denominations = {1000, 500, 200, 100, 50, 20, 10, 5};
    double remainder = money;
    int[] counter = new int[denominations.length];

    // calculate each denomination
    for(int currentDenomination = 0; currentDenomination < denominations.length; currentDenomination++) {
        counter[currentDenomination] = (int) (remainder / denominations[currentDenomination]);
        remainder = remainder % denominations[currentDenomination];
    }

    // print non-zero denominations
    for(int currentDenomination = 0; currentDenomination < denominations.length; currentDenomination++) {
        if(counter[currentDenomination] > 0){
            System.out.println("AED " + (int)denominations[currentDenomination] + ": " + counter[currentDenomination]);
        }
    }
}
```

→ The denomination method is my helper method, as it helps calculate and print the denomination breakdown, a common task that is recurring throughout the code.

Limitations:

- One of the limitations is that my program is unable to produce a “Not Found” prompt when the user is using the menu’s option 1 and searching for an individual in the database. Instead of prompting the user that the person they looked up is “Not Found”, the program would crash or skip the task and return the user to the menu.

```
--------------------------------------------------------------------------------------------
Please enter your desired function:
1. Enter a name and display change to be given for each denomination.
2. Find the name with the smallest amount and display change to be given for each denomination.
3. Find the name with the largest amount and display change to be given for each denomination.
4. Calculate and display the total number of currency note for each denomination.
5. Calculate and display the total amount for the sum of all denominations.
6. Exit
--------------------------------------------------------------------------------------------
John
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:947)
        at java.base/java.util.Scanner.next(Scanner.java:1602)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2267)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2221)
        at assignment.ChangeC.currencyMenu(ChangeC.java:136)
        at assignment.ChangeC.main(ChangeC.java:56)
C:\Users\tomah\AppData\Local\NetBeans\Cache\19\executor-snippets\run.xml:111: The following error occurred while executing this line:
C:\Users\tomah\AppData\Local\NetBeans\Cache\19\executor-snippets\run.xml:68: Java returned: 1
BUILD FAILED (total time: 32 seconds)
```

```
--------------------------------------------------------------------------------------------
Please enter your desired function:
1. Enter a name and display change to be given for each denomination.
2. Find the name with the smallest amount and display change to be given for each denomination.
3. Find the name with the largest amount and display change to be given for each denomination.
4. Calculate and display the total number of currency note for each denomination.
5. Calculate and display the total amount for the sum of all denominations.
6. Exit
--------------------------------------------------------------------------------------------
1
Please enter name
John
--------------------------------------------------------------------------------------------
Please enter your desired function:
1. Enter a name and display change to be given for each denomination.
2. Find the name with the smallest amount and display change to be given for each denomination.
3. Find the name with the largest amount and display change to be given for each denomination.
4. Calculate and display the total number of currency note for each denomination.
5. Calculate and display the total amount for the sum of all denominations.
6. Exit
--------------------------------------------------------------------------------------------
```

- The other limitation of my program is that when a user has chosen to manually input data, upon entering both their name and the amount of money they wish to input into the system, enter must be pressed twice after inputting the money value for the code to move forward.

```
run:
Student Name: Styverson Ng
Student Number: 35427675
Murdoch University
Lecturer's Name: Siew Cheong Chong
Please enter the name of the person:
Charles
Please enter the currency note value for the person:
5675

Do you have more person to enter (Y/N)
```

- Another limitation of my program is that while the addition of new person's details should be at least 10, the array that I have is only limited to 10. Although the limit can be altered, it has to be done manually.

```
public class ChangeC {
    public static void main(String[] args){
        StudentInfo();
        Scanner input = new Scanner(source: System.in);
        int arrayCapacity = 10;
        Person[] people = new Person[arrayCapacity];
        boolean usePreset = false; // change to true
```

Testing:

Test 1: Test for getSample readings

1) Firstly, the instruction to hardcode some data for the lecture to use was given
2) Next, I have hardcoded 10 different sets of data to be read
3) To test if these data is readable and can be used to retrieve data from the rest of the code, I coded a print function that displays the hardcoded arrays

```
public static void getSample(Person[] data){
    //Hard code some instances to facilitate testing
    //Hard coding for lecturer's use
    data[0] = new Person(personName: "Charles",personMoney:4685);
    data[1] = new Person(personName: "Carlos",personMoney:3265);
    data[2] = new Person(personName: "Max",personMoney:6985);
    data[3] = new Person(personName: "Pierre",personMoney:6635);
    data[4] = new Person(personName: "Daniel",personMoney:10180);
    data[5] = new Person(personName: "Fernando",personMoney:1925);
    data[6] = new Person(personName: "Lewis",personMoney:685);
    data[7] = new Person(personName: "Oscar",personMoney:3215);
    data[8] = new Person(personName: "Lando",personMoney:7345);
    data[9] = new Person(personName: "George",personMoney:965);
}

    // method to display person(s) information
public static void printInfo(Person[] person){
    System.out.println(x: "Customer:");
    for (Person currentPerson : person){
        if (currentPerson != null){
            System.out.println(x: currentPerson);
        }
    }
}
```

```
run:
Student Name: Styverson Ng
Student Number: 35427675
Murdoch University
Lecturer's Name: Siew Cheong Chong
-----------------------------------------------------------------------------------------
Customers:
Charles AED 4685.0
Carlos AED 3265.0
Max AED 6985.0
Pierre AED 6635.0
Daniel AED 10180.0
Fernando AED 1925.0
Lewis AED 685.0
Oscar AED 3215.0
Lando AED 7345.0
George AED 965.0
-----------------------------------------------------------------------------------------
Please enter your desired function:
1. Enter a name and display change to be given for each denomination.
2. Find the name with the smallest amount and display change to be given for each denomination.
3. Find the name with the largest amount and display change to be given for each denomination.
4. Calculate and display the total number of currency note for each denomination.
5. Calculate and display the total amount for the sum of all denominations.
6. Exit
-----------------------------------------------------------------------------------------
```

Test 2: Test for manual user input of data loop

1) First, I needed to ensure that the program is able to read both the preset and manually set data without disrupting the rest of the code, hence an if-each statement where a boolean usePreset can be assigned as true or false.
2) Afterwards, I initialize “index” to track the position of the person in the array and “cont” to track the addition option for a new person
3) Using a while loop, in consideration of the user’s infinite desire to enter people, the user inputs both name and money values into the system and is then prompted if he/she wishes to enter more person’s detail using (Y/N) as their answer.
4) Once the system has received the array capacity (limitation) or n(case-insensitive) as an answer from the user, the loop will break and execute the currency menu for the user to use.

```java
public class ChangeC {
    public static void main(String[] args){
        StudentInfo();
        Scanner input = new Scanner(source: System.in);
        int arrayCapacity = 10;
        Person[] people = new Person[arrayCapacity];
        boolean usePreset = false; // change to true to use getSample method for preset hardcoding sameples

        // hardcoding sample data for testing and lecturer's use
        if(usePreset == true){
            getSample(data:people);
            // uncomment printInfo to check if getSample's data has been registered into the array
            //printInfo (people);
        }
        else{
            int index = 0; // used to track the position of a person in the array
            boolean cont = true;

            // loop to insert person's name and detail
            while(cont == true && index < arrayCapacity){
                String name = getPersonName(input, person: people);
                double money = getPersonMoney(input);
                input.nextLine();

                Person newPerson = new Person(personName: name,personMoney:money);
                newPerson.setName(name);
                newPerson.setMoney(money);
                people[index++] = newPerson; // add the new person and increment index

                // option for user to add more person or break loop
                if (index < arrayCapacity){
                    System.out.println(x: "Do you have more person to enter (Y/N)");
                    String reply = input.nextLine();

                    if(reply.equalsIgnoreCase(anotherString: "n")){
                        cont = false;
                    }
                }
            }
        }
        // calling menu application for the user to choose
         currencyMenu(input, people);
    }
```

```
run:
Student Name: Styverson Ng
Student Number: 35427675
Murdoch University
Lecturer's Name: Siew Cheong Chong
Please enter the name of the person:
Charles
Please enter the currency note value for the person:
2345

Do you have more person to enter (Y/N)
y
Please enter the name of the person:
Max
Please enter the currency note value for the person:
9845

Do you have more person to enter (Y/N)
n
-------------------------------------------------------------------------------------------
Please enter your desired function:
1. Enter a name and display change to be given for each denomination.
2. Find the name with the smallest amount and display change to be given for each denomination.
3. Find the name with the largest amount and display change to be given for each denomination.
4. Calculate and display the total number of currency note for each denomination.
5. Calculate and display the total amount for the sum of all denominations.
6. Exit
-------------------------------------------------------------------------------------------
```

Test 3: Test for non-zero outputs in denomination

1) First, I initialized initialAmount and remainder to use for my calculation. Only to realize that although I can obtain the denomination outcome, I am unable to single out and detect the zero outputs without breaking the rest of the method.
2) Next, I created a denomination array with the denominations provided and a counter array to go through the denomination array one by one.
3) Hence, the system will go through the money data in the people array with the descending denomination and its remainder.
4) By dividing the original amount by the first denomination, the number of that denomination will return, leaving a remainder. Using that remainder % the same denomination we get a new remainder and the process repeats.
5) If the output of remainder is less than 1 and by forcing it into an integer, which will be read as zero, it will be a zero-value output and the system will go through each iteration of the process above and only print non-zero outputs.

```java
// method for calculating the denominations of customer's money
public static void denomination(double money){
    double[] denominations = {1000, 500, 200, 100, 50, 20, 10, 5};
    double remainder = money;
    int[] counter = new int[denominations.length];

    // calculate each denomination
    for(int currentDenomination = 0; currentDenomination < denominations.length; currentDenomination++) {
        counter[currentDenomination] = (int) (remainder / denominations[currentDenomination]);
        remainder = remainder % denominations[currentDenomination];
    }

    // print non-zero denominations
    for(int currentDenomination = 0; currentDenomination < denominations.length; currentDenomination++) {
        if(counter[currentDenomination] > 0){
            System.out.println("AED " + (int)denominations[currentDenomination] + ": " + counter[currentDenomination]);
        }
    }
}
```

```
-----------------------------------------------------------------------------------------------
Please enter your desired function:
1. Enter a name and display change to be given for each denomination.
2. Find the name with the smallest amount and display change to be given for each denomination.
3. Find the name with the largest amount and display change to be given for each denomination.
4. Calculate and display the total number of currency note for each denomination.
5. Calculate and display the total amount for the sum of all denominations.
6. Exit
-----------------------------------------------------------------------------------------------
2
-----------------------------------------------------------------------------------------------
Person with the smallest amount of money:
Lewis AED 685.0
AED 500: 1
AED 100: 1
AED 50: 1
AED 20: 1
AED 10: 1
AED 5: 1
-----------------------------------------------------------------------------------------------
```

Test Table

Test ID	Test description/justification – what is the test for and why this particular test	Actual data for this test	Expected output	Actual desk check result when desk check is carried out	Desk check outcome – Pass/Fail
T1	Test if the preset array can be read	usePreset = true printInfo	all preset person data are displayed	all 10 preset persons displayed correctly	Pass
T2	Test if user can enter new person's name and money	usePreset = false; Name: "John" Money: 5000	"Do you have more person to enter (Y/N)"; Details of John is stored	John's data stored correctly, prompt user for more entries	Pass
T3	Test array size for unlimited entries based on user's need	usePreset = false; Name: "(11 names)" Money: (11 values)	"Do you have more person to enter (Y/N)"; Details of all 11 persons is stored	11 persons data stored correctly, prompt user for more entries	Pass
T4	Test for duplicate name in array	usePreset = true Name: "Charles"	"You have entered a registered name. Please enter a different name."	Prompt user to enter a different name due to duplicate	Pass
T5	Test for non-multiple of 5 money input	Money: 1234	"Please enter the amount in multiple of 5."	Prompt user to enter a value in multiples of 5	Pass
T6	Test currency menu option 1 - display change for name	Option: 1; Name: "Charles"	Denomination breakdown of 4685 AED	Display Charles, 4685 AED & denomination breakdowns	Pass
T7	Test currency menu option 2 - smallest amount	Option: 2	Name of person with smallest amount and	Display Lewis, 685 AED &	Pass

			denomination breakdown	denomination breakdowns	
T8	Test currency menu option 3 - largest amount	Option: 3	Name of person with largest amount and denomination breakdown	Display Daniel, 10180 AED & denomination breakdowns	Pass
T9	Test currency menu option 4 - total notes count	Option: 4	Denomination breakdown of total amount of notes	Display correct total amount denomination breakdown	Pass
T10	Test currency menu option 5 - total amount	Option: 5	Total amount of money in array	Display correct total amount	Pass
T11	Test currency menu exit	Option: 6	exit message	Display "Thank you and Goodbye!"	Pass

Results of Program Testing

Test ID	Test description/justification – what is the test for and why this particular test	Actual data for this test	Expected output	Actual desk check result when desk check is carried out	Test run outcome – Pass/Fail
T1	Test if the preset array can be read	usePreset = true printInfo	all preset person data are displayed	all 10 preset persons displayed correctly	Pass
T2	Test if user can enter new person's name and money	usePreset = false; Name: "John" Money: 5000	"Do you have more person to enter (Y/N)"; Details of John is stored	John's data stored correctly, prompt user for more entries	Pass
T3	Test array size for unlimited entries based on user's need	usePreset = false; Name: "(11 names)" Money: (11 values)	"Do you have more person to enter (Y/N)"; Details of all 11 persons is stored	Array size has been set to 10. The menu is automatically prompted to the user	Fail
T4	Test for duplicate name in array	usePreset = true Name: "Charles"	"You have entered a registered name. Please enter a different name."	Prompt user to enter a different name due to duplicate	Pass
T5	Test for non-multiple of 5 money input	Money: 1234	"Please enter the amount in multiple of 5."	Prompt user to enter a value in multiples of 5	Pass
T6	Test currency menu option 1 - display change for name	Option: 1; Name: "Charles"	Denomination breakdown of 4685 AED	Display Charles, 4685 AED & denomination breakdowns	Pass

T7	Test currency menu option 2 - smallest amount	Option: 2	Name of person with smallest amount and denomination breakdown	Display Lewis, 685 AED & denomination breakdowns	Pass
T8	Test currency menu option 3 - largest amount	Option: 3	Name of person with largest amount and denomination breakdown	Display Daniel, 10180 AED & denomination breakdowns	Pass
T9	Test currency menu option 4 - total notes count	Option: 4	Denomination breakdown of total amount of notes	Display correct total amount denomination breakdown	Pass
T10	Test currency menu option 5 - total amount	Option: 5	Total amount of money in array	Display correct total amount	Pass
T11	Test currency menu exit	Option: 6	exit message	Display "Thank you and Goodbye!"	Pass